



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Algoritmos e Estrutura de Dados I

Lista 2.1 – Tipo Abstrato de Dados

- 1) Elabore um tipo abstrato de dados para representar uma matriz contendo os seguintes campos: quantidade de linhas, quantidade de colunas e os elementos da matriz. A estrutura para representar o TAD deve ter o seguinte formato:

```
struct matriz {  
    int l;  
    int c;  
    int **m;  
}
```

As operações devem ser:

- Criar a matriz (alocar memória dinamicamente)
- Preencher a matriz
- Imprimir os dados da matriz;
- Somar duas matrizes

Na main, utilize o TAD matriz para criar duas matrizes A e B, 2 x 2, com dados digitados no teclado e imprimir a matriz A, B e a matriz resultante de A + B.

- 2) Crie um TAD para cada cenário abaixo. Teste cada TAD criado no programa principal.
- a) Uma aplicação para gerenciar informações sobre n alunos contendo os seguintes dados: código (inteiro), nome (cadeia de caracter de tamanho máximo = 100) e uma lista de notas (números reais) de tamanho fixo = 5. As operações do TAD são:
- Preencher as informações dos n alunos;
 - Retornar a média de um determinado aluno dado sua matrícula;
 - Retornar a média das médias de todos os alunos.
- A lista de n alunos deve ser alocada dinamicamente na função principal.
- b) Uma aplicação para gerenciar informações sobre n carros que ficam em um estacionamento: placa, marca, modelo e cor. A aplicação deve permitir:
- Alocar uma lista de n carros.
 - Preencher as informações dos n carros.
 - Alterar as informações de um carro com uma determinada placa.
 - Imprimir os dados de um carro com uma determinada placa
 - Retornar a quantidade de veículos de uma marca
- c) Um tipo para representar conjuntos de números inteiros. Seu tipo abstrato deverá armazenar os elementos do conjunto e o seu tamanho n. Os elementos devem ser armazenados em um vetor. O valor de n deve ser fornecido pelo usuário. Seu TAD deve possuir funções para: a.



MINISTÈRIO DA EDUCAÇÃO UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

- criar um conjunto vazio;
 - preencher os dados de um conjunto;
 - criar um novo conjunto através da união de dois conjuntos;
 - criar um novo conjunto através da interseção de dois conjuntos;
 - verificar se dois conjunto são iguais (possuem os mesmos elementos), se for, retornar 1, caso contrário, retorne 0;
 - imprimir um conjunto;
- d) Uma aplicação para gerenciar informações sobre um grupo de pessoas contendo os seguintes dados: código, nome, endereço e telefone. As operações do TAD são:
- Alocar dinamicamente uma lista de n pessoas;
 - Incluir uma pessoa na lista
 - i. Você deve definir uma estratégia para determinar se a lista não está cheia. Se a pessoa puder ser incluída na lista, a operação deve retornar 1. Caso contrário, se a lista estiver cheia, a função deve retornar 0.
 - Excluir uma pessoa da lista
 - Imprimir os dados da pessoa com um determinado código.

Para cada TAD, deve ser criada a especificação e a implementação das operações. Em C, isto feito em arquivos separados. A especificação em um arquivo .h e a implementação em um arquivo .c. A especificação do TAD deve conter:

- Nome do TAD
- Dados
- Operações

Para os dados, é necessário definir a estrutura que deverá ser usada para representar estes dados. Já para cada uma das operações, basta que seja definida sua assinatura (nome da operação, tipo de retorno e parâmetros de entrada).

Ao alocar uma estrutura de forma dinâmica, caso uma de suas partes seja um vetor ou uma matriz dinâmica, a alocação dos elementos do vetor e/ou da matriz deve ser feita separadamente. Por exemplo, considere a struct abaixo:

```
struct exemplo{  
    int x;  
    int *p;}
```

Ao alocar memória para uma estrutura do tipo struct exemplo, você deve fazer:

Seja v e n duas variáveis:

```
struct exemplo *v;
```

```
int n; // número de elementos do vetor p que compõe a estrutura
```

A alocação de v deve ser feita desta forma:

```
v = (struct exemplo*) malloc(sizeof(struct exemplo)); // alocação de v
```

```
v->p=(int *) malloc(sizeof(int) * n); // alocação do campo p da estrutura
```